

UNIVERSIDADE FEDERAL DE OURO PRETO - UFOP

ORGANIZAÇÃO DE COMPUTADORES

TRABALHO PRÁTICO I

Grupo: Luisa Ribeiro Notaro
Marcos Vinicio Euzébio

Docente: Pedro Henrique Lopes Silva

Ouro Preto
2023

Introdução.....	3
Desenvolvimento.....	3
Comandos utilizados.....	3
Insertion Sort.....	4
Selection Sort.....	6
Conclusão.....	7

Introdução

O docente orientador deste Segundo Trabalho Prático da disciplina de Organização de Computadores solicitou que fossem implementados os algoritmos de *Insertion Sort* e *Selection Sort* na linguagem de montagem MIPS. O objetivo deste relatório é elucidar a linha de raciocínio utilizada pelo grupo para a implementação desses algoritmos.

Desenvolvimento

Comandos utilizados

Um conjunto não muito extenso de comandos foi utilizado para a implementação dos algoritmos de ordenação, ei-los:

- **add:** recebe três registradores \$t1, \$t2 e \$t3 quaisquer. Soma o conteúdo de \$t2 a \$t3 e salva em \$t1.
- **addi:** recebe dois registradores \$t1 e \$t2 e um valor x quaisquer. Soma o conteúdo de \$t2 ao valor x e armazena o resultado em \$t1.
- **li:** recebe um registrador \$t1 e um valor x quaisquer. Salva o valor x no registrador \$t1.
- **sb:** recebe um registrador \$t1 qualquer e uma posição na memória efetiva (RAM). Salva o valor contido no registrador \$t1 nessa posição da memória efetiva.
- **beq:** vai para o rótulo especificado como terceiro argumento caso os valores dos dois primeiros argumentos sejam iguais.
- **bne:** comando similar ao “beq”. Se difere do seguinte: para que ele encaminhe para o rótulo especificado, os dois primeiros argumentos precisam ter valores diferentes ao invés de iguais.
- **lw:** realiza o efeito contrário ao comando “sb” visto anteriormente. Recebe um valor de uma posição da memória efetiva e o armazena em um registrador \$t1 qualquer.
- **sgt:** recebe três registradores \$t1, \$t2 e \$t3 quaisquer. Se o valor de \$t2 é maior que o de \$t3, \$t1 é igual a 1, caso contrário, é igual a zero.
- **j:** realiza um salto até algum rótulo.

Para além dos comandos colocados acima, foram utilizados rótulos, que irão ser demonstrados conforme os códigos forem sendo explicados.

Um adendo importante a ser comentado é que o endereço de memória efetiva utilizado foi por meio do registrador “\$sp”, que armazena o último endereço de memória efetiva disponível. Portanto, é como se as coisas fossem feitas de “baixo para cima” ou “da direita para a esquerda”, isto é, o registrador “\$sp” guardará o primeiro endereço de memória utilizado.

Insertion Sort

Abaixo, há o código referente ao *Insertion Sort*.

```
add $s0, $zero, $sp

li $t0, 2
sb $t0, ($s0)
li $t0, 7
sb $t0, -4($s0)
li $t0, 8
sb $t0, -8($s0)
li $t0, 3
sb $t0, -12($s0)
li $t0, 4
sb $t0, -16($s0)
li $t0, 1
sb $t0, -20($s0)
li $t0, 6
sb $t0, -24($s0)
li $t0, 5
sb $t0, -28($s0)

li $t3, 8      # n = 8
li $t2, 1      # i = 1

# t1 = j
# t2 = i
# t3 = n
# t4 utilizado como flag

LOOP_E:
    beq $t2, $t3, END_LOOP_E      # if i == n, sai do loop
    add $t1, $zero, $t2           # j == i inicialmente
    add $s1, $zero, $s0           # $s1 eh um registro auxiliar para andar na
    # memoria no loop interno
    LOOP_I:
        beq $t1, $zero, END_LOOP_I    # if j == 0
        lw $s6, -4($s1)
        lw $s7, 0($s1)
        sgt $t4, $s6, $s7
        beq $t4, $zero, SWITCH        # if v[j] < v[j - 1]
        bne $t4, $zero, END_LOOP_I    # if v[j] > v[j - 1] quebra o loop
        SWITCH:
```

```

interno
    AFTER_SWTICH:
        addi $s1, $s1, 4
        addi $t1, $t1, -1
        j LOOP_I
    END_LOOP_I:
    # as linhas abaixo equivalem aa i++
    addi $s0, $s0, -4
    addi $t2, $t2, 1
    j LOOP_E

SWITCH:
    sb $s6, 0($s1)
    sb $s7, -4($s1)
    j AFTER_SWTICH

END_LOOP_E:

```

Código 1: Insertion Sort feito em MIPS.

Da primeira linha até o rótulo “LOOP_E” estão sendo feitas apenas algumas inserções na memória efetiva, que representará um vetor desordenado. Além disso, inicializa-se “i” com 1 e “n” com 8. Observe que os seguintes valores foram respectivamente inseridos: 2 7 8 3 4 1 6 5. A sequência é composta de oito valores por motivos de simplificação.

Dentro do rótulo “LOOP_E” que simula um *looping*, é feita uma verificação: se o índice i - que possui o valor representado pelo registrador \$t2 - é igual ao valor do índice n - que possui valor representado pelo registrador \$t3. Ora, se são iguais, o programa é finalizado, porque, caso contrário, extrapolaria-se o limite do “vetor”. Caso contrário, criamos um índice “j”, cujo valor é representado pelo registrador \$t1. Note que, com a criação do índice “j”, torna-se necessário criar um outro valor auxiliar para que seja possível andar na memória sem perder a referência para a posição onde o valor “i” está; quem realiza essa função é o registrador “\$s1”.

Para um fim objetivo, ilustrativo e elucidativo, toda vez que for falado sobre os índices “i”, “j” e “n”, tenha em mente que, na prática, está sendo falado dos registradores \$t2, \$t1 e \$t3, respectivamente.

Mais internamente ao rótulo “LOOP_E” (que simula um *looping* também), tem-se o rótulo “LOOP_I”. Dentro deste, algumas operações são realizadas. Primeiramente, verifica-se se o índice “j” é igual ao valor zero, se sim, chegou-se ao

início do vetor e não é mais necessário a iteração dentro do *looping*. Caso contrário, a posição mais à esquerda da atual (para onde “j” aponta), é comparada com esta. Se o valor da posição atual é maior que o valor da posição à sua direita, o comando *beq* manda o fluxo de execução do código para o rótulo “SWITCH” o qual realiza as trocas. Depois de ter feito a troca, o índice “j” anda e o processo é repetido. O processo é repetido até que se chegue ao final do vetor ou o valor da posição atual já seja menor que o valor à direita daquela posição.

Selection Sort

Abaixo, há o código referente ao *Selection Sort*.

```
add $s0, $zero, $sp

li $t0, 2
sb $t0, ($s0)
li $t0, 7
sb $t0, -4($s0)
li $t0, 8
sb $t0, -8($s0)
li $t0, 3
sb $t0, -12($s0)
li $t0, 4
sb $t0, -16($s0)
li $t0, 1
sb $t0, -20($s0)
li $t0, 6
sb $t0, -24($s0)
li $t0, 5
sb $t0, -28($s0)

li $t3, 8    # n = 8
li $t2, 0    # i = 1

LOOP_E:
    beq $t2, $t3, END_LOOP_E    # if i == n quebra o loop
    lw $t9, 0($s0)              # t9 armazena o valor contido na primeira
posicao daquela iteracao
    lw $t7, 0($s0)              # t7 representa o menor valor
    addi $t1, $t2, 1            # j = i + 1
    addi $s1, $s0, -4           # s1 eh um registrador auxiliar que anda na
memoria
    add $s3, $zero, $s0         # s3 eh outro registrador auxiliar para
armazenar uma posicao na memoria (funciona como o k)
    LOOP_I:
        beq $t1, $t3, END_LOOP_I    # if j == n quebra o loop
        lw $t8, 0($s1)              # t8 representa valores concorrentes para
menor valor
        bge $t8, $t7, CONTINUE
        add $s3, $zero, $s1         # grava no registrador $s3 o indice de
maior valor na memoria
```

```

        add $t7, $zero, $t8
        CONTINUE:
        addi $s1, $s1, -4
        addi $t1, $t1, 1
        j LOOP_I
    END_LOOP_I:
    j SWITCH                                # realiza a troca necessaria
    AFTER_SWITCH:
    # as linhas abaixo equivalem aa i++
    addi $s0, $s0, -4
    addi $t2, $t2, 1
    j LOOP_E
SWITCH:
    sb $t7, 0($s0)
    sb $t9, 0($s3)
    j AFTER_SWITCH
END_LOOP_E:

```

Código 2: Selection Sort feito em MIPS.

Ele possui um fluxo similar ao Insertion Sort. O registrador \$t7 sempre armazenará o índice cujo valor para o qual aponta será comparado com os demais elementos. Se algum elemento de outra posição do vetor, quando comparada à ele, for menor, ele recebe o valor presente naquela outra posição do vetor. Assim como a referência de menor valor na memória efetiva é apontada para aquele novo local.

Conclusão

Conclui-se que, embora seja um trabalho que não possui relação com os anteriores, ele é excelente para a introdução sobre a linguagem de montagem, pois oferece uma noção de como as coisas funcionam “por baixo dos panos”, isto é, como um compilador funciona ou se parece.